2026 15th International Conference on Modern Circuits and Systems Technologies (MOCAST) | 979-8-3195-1875-0/26/$31.00 ©2026 IEEE | DOI: 10.1109/MOCAST70204.2026.11626365

FPGA Implementation of a Hybrid Chaotic True Random Number Generator Using Fibonacci Ring Oscillator

Mohamed E. Mohamed[1], Wafaa S. Sayed[2], Ahmed H. Madian[1,3], Lobna A. Said[1]

[1]Nanoelectronics Integrated Systems Center (NISC), Nile University, 12588, Giza, Egypt
[2]Engineering Mathematics and Physics Department, Faculty of Engineering, Cairo University, 12613, Giza, Egypt
[3]Radiation Engineering Department, National Centre for Radiation Research and Technology, Egyptian Atomic Energy Authority, 11787, Cairo, Egypt

***Abstract*—This paper presents a fully digital true random number generator (TRNG) combining the jitter of Fibonacci Ring Oscillators (FiROs) with the complex dynamics of chaotic maps. The ring oscillators are employed to introduce intrinsic noise into the chaotic maps, effectively breaking their deterministic behavior and enhancing the overall entropy of the generated sequence. Two specific chaotic systems, the Logistic map and the Tent map, are evaluated to increase the entropy of the raw signal while suppressing deterministic patterns. The system is implemented on a Zynq-7000 FPGA and evaluated using NIST SP 800-90B, NIST SP 800-22, and AIS 31 test suites, successfully passing all applied statistical tests. According to the entropy assessment methodology of NIST SP 800-90B, the achieved min-entropy values are 7.987 bits/sample for the Logistic map and 7.953 bits/sample for the Tent map. Furthermore, the architecture achieves a throughput of 142.5 Mbps, meeting the requirements of secure embedded systems.**

***Index Terms*—Chaotic maps, FPGA, Hardware security, Jitter, Ring Oscillator, TRNG**

I. Introduction

True Random Number Generators (TRNGs) are fundamental to modern cryptography as they provide unpredictable random bits used for key generation, encryption schemes, authentication protocols, and secure communications [1]. Pseudo-Random Number Generators (PRNGs) are fast and can generate large sequences of numbers efficiently. However, they rely on deterministic algorithms and a seed, making them predictable if the seed is known. In contrast, TRNGs extract randomness from physical processes, producing unpredictable sequences with stronger security.

Conventional TRNGs often exploit analog sources such as thermal or phase noise [2], but these approaches require calibrated analog circuitry and are sensitive to process, voltage, and temperature variations. This has motivated growing interest in fully digital TRNG implementations on Field-Programmable Gate Arrays (FPGAs), which offer a flexible and scalable hardware platform [3]. A typical digital TRNG consists of an entropy source, a digitization stage, and a post-processing unit to mitigate bias and correlations. Among digital entropy sources, ring oscillators (ROs) are widely adopted due to their simplicity and low hardware cost. However, single or lightly combined RO-based designs may suffer from limited entropy, output correlation, and reduced robustness under environmental variations [4].

These challenges have motivated research efforts to improve entropy quality without excessive overhead. Several architectural strategies have been proposed to address these issues. In [5], braided and hybrid cross-coupled logic gates were used to increase phase diversity and reduce correlation between internal signals. Other works have exploited built-in FPGA modules as entropy sources to avoid the need for dedicated oscillator structures. In [6], Digital Clock Managers (DCMs) in Xilinx devices were used to induce controlled metastability through dynamic phase shifting. Similarly, in [7], DSP blocks were used as multiple ring oscillators by introducing feedback and configuring arithmetic logic units for bitwise XOR operations, enabling entropy extraction without dedicated oscillator circuitry.

This work proposes a fully digital FPGA-based TRNG that combines a Fibonacci ring oscillator with a lightweight chaotic enhancement stage and a simple correction mechanism to reduce bias and correlation. Unlike previous works that focus on exploiting FPGA resources or modifying ring oscillator architectures, our approach exploits the unpredictable behavior of Logistic and Tent maps to enhance the oscillator's entropy, increasing the randomness of the generated sequence without introducing complex circuitry or excessive resource overhead. The resulting design achieves high-quality entropy at high throughput, providing strong, reliable randomness suitable for practical embedded applications.

II. System Architecture and Design Methodology

A. Entropy Source: Fibonacci Ring Oscillators

The standard ring oscillator simply cascades inverters in a loop, while a FiRO introduces a feedback network inspired by Linear Feedback Shift Registers (LFSRs). This modification disrupts the regular periodicity often observed in simple loops, reduces dependencies on placement and routing, and improves jitter accumulation [8].

The FiRO in Fig. 1, replaces synchronous flip-flops with asynchronous inverter stages, where signal propagation is entirely governed by gate delays. These delays fluctuate due to thermal noise and supply voltage variations (PVT), forming the core source of unpredictability.

The feedback structure is defined by a characteristic polynomial

$$P(x) = \sum_{i=0}^{r} f_i x^i, \tag{1}$$

979-8-3195-1875-0/26/$31.00 ©2026 IEEE

Authorized licensed use limited to: NATIONAL INSTITUTE OF TECHNOLOGY KURUKSHETRA. Downloaded on October 01,2026 at 10:49:06 UTC from IEEE Xplore. Restrictions apply.

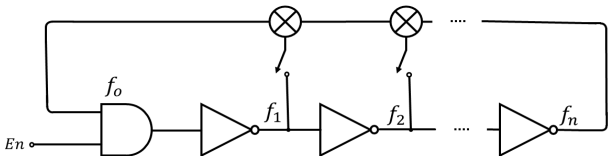


Fig. 1: Fibonacci Ring Oscillator structure.

where the coefficients f_i determine the tap points. To ensure reliable oscillation and avoid fixed-point states, the polynomial must satisfy:

$$P(x) = (x+1)h(x), \quad h(1) = 1. \tag{2}$$

In the proposed implementation, four independent FiROs are employed to enhance entropy diversity. The first and third FiROs use polynomial $x^5+x^4+x^2+1$, while the second and fourth use polynomial x^5+x^3+x+1.

Each FiRO provides two tapped outputs. The outputs from the four oscillators are combined to form an 8-bit entropy vector, which serves as the input to the enhancement stage.

B. Entropy Enhancement and Post-Processing

Although FiROs provide raw physical randomness, their direct output may still contain bias or residual correlations [4]. To address this, the 8-bit entropy vector is processed using digital chaotic maps, followed by a lightweight enhancement stage.

Chaotic systems are highly sensitive to initial conditions [9], a property quantified by the Lyapunov exponent (LE). A positive LE indicates exponential divergence of nearby trajectories, which is a signature of chaos. Two chaotic maps are implemented; the Logistic map given by:

$$x_{n+1} = r\,x_n(1-x_n), \quad x_n \in (0,1), \quad r \in [0,4], \tag{3}$$

and the Tent map given by:

$$x_{n+1} = \begin{cases} ax_n, & 0 \le x_n < 0.5, \\ a(1-x_n), & 0.5 \le x_n \le 1, \end{cases} \quad 1 < a < 2, \tag{4}$$

where x_n is the state variable, while r and a are the control parameters. Both maps exhibit positive LE values within their chaotic regimes as shown in Fig. 2.

Since finite precision in digital implementations may cause short periodic cycles, a noise injection technique is employed. At every iteration, the chaotic state variable is perturbed using the 8-bit entropy vector from the previous stage of FiRos [4]. This continuous perturbation forces transitions between different trajectories, suppresses periodic windows, and significantly expands the effective chaotic regime. Figure 3 shows the improvement in state distribution with noise injection.

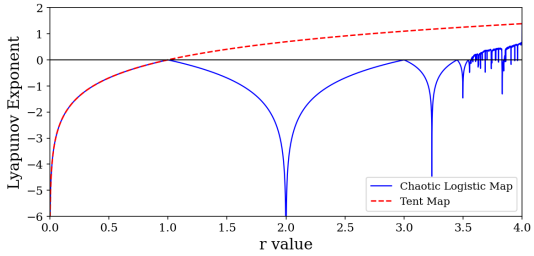


Fig. 2: Lyapunov exponent of the Logistic and Tent maps.

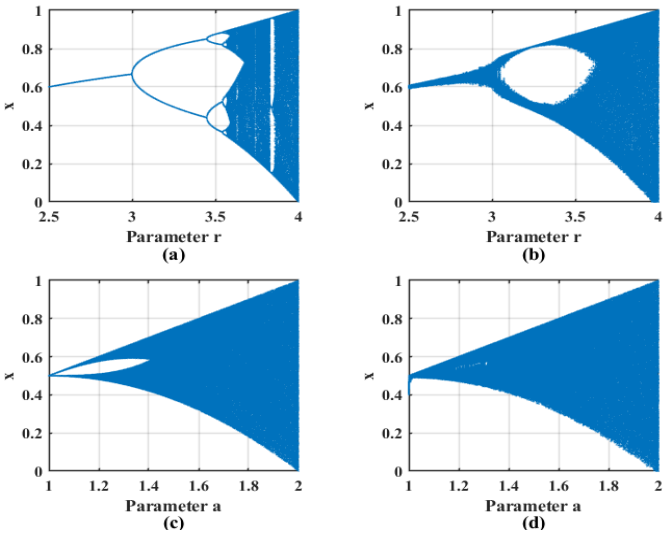


Fig. 3: Bifurcation diagrams of the chaotic maps: (a) Logistic, (b) noisy Logistic, (c) Tent, and (d) noisy Tent.

To further enhance statistical balance, a lightweight correction mechanism is applied at the output stage. First, the 8-bit chaotic output is reduced to a single bit using an XOR operation. A signed counter then monitors the disparity between generated '1's and '0's within a predefined range (e.g., $[-4,+4]$). If a boundary is reached, the next bit is conditionally inverted; otherwise, it passes unchanged. This further enhancement stage mitigates residual bias while keeping hardware overhead minimal.

III. Hardware Realization

The overall hardware architecture is illustrated in Fig. 4, integrating the entropy source, the chaotic processing core, the correction scheme, and the output buffering stages. The proposed TRNG is implemented on the Zynq-7000 Zybo Z7-10 board, as shown in Fig. 5. Both the logistic and tent maps are implemented using an 8-bit unsigned fixed-point format, ensuring low-complexity hardware realization while preserving the nonlinear dynamics required for high-entropy generation.

The correction scheme output bit is sampled by a D flip-flop driven by a 142.5 MHz on-chip PLL, accumulated in an 8-bit shift register, and stored in a FIFO. The buffered data are serialized using a UART transmitter, and the UART signals are

Authorized licensed use limited to: NATIONAL INSTITUTE OF TECHNOLOGY KURUKSHETRA. Downloaded on October 01,2026 at 10:49:06 UTC from IEEE Xplore. Restrictions apply.

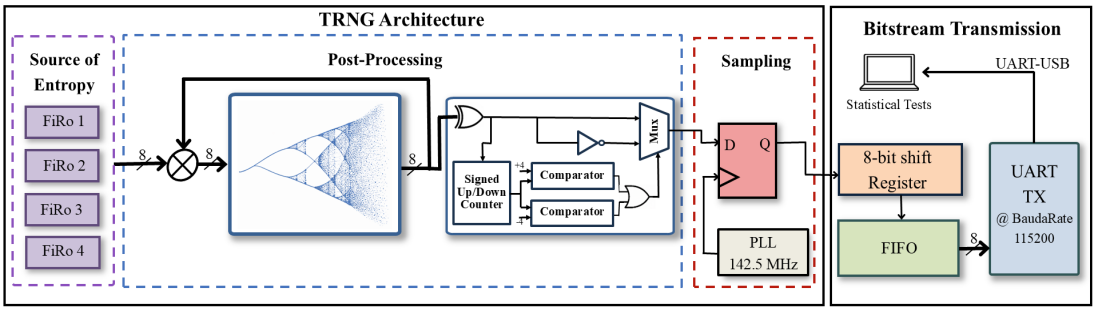


Fig. 4: Hardware architecture of the proposed TRNG.

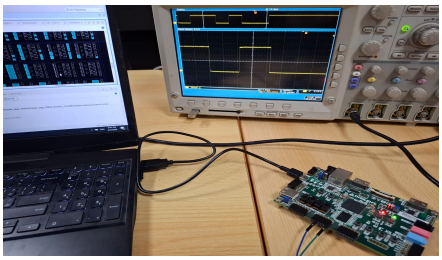


Fig. 5: Experimental setup for the proposed TRNG.

converted to USB format through a UART-USB module before being transmitted to the host PC for statistical evaluation.

IV. Experimental Results

A total of 10 MByte of random data is generated and collected for offline analysis. The output data is evaluated using the NIST SP 800-90B, NIST SP 800-22, and AIS 31 test suites, which together provide a comprehensive assessment of the entropy characteristics and randomness quality of the generated sequence.

A. NIST SP 800-90B Entropy Evaluation

The NIST SP 800-90B standard provides a framework for evaluating the quality of random bit generators by focusing on min-entropy (H_∞), which measures the worst-case predictability of the source. The primary objective is to verify that the bits are Independent and Identically Distributed (IID), meaning that each bit is not influenced by previous values and originates from the same probability distribution [10].

From Table I, both chaotic maps successfully pass the IID validation track. The p-values for the Chi-square tests and the successful completion of the Longest Repeated Substring (LRS) test indicate that the sequences show no clear patterns or statistical dependencies. In addition, the Logistic map gives a min-entropy of 0.9984 bits per bit, slightly outperforming the Tent map's 0.9942 bits/bit. These results confirm that both maps provide strong entropy suitable for cryptographic applications, with the Logistic map offering a slightly higher security margin.

In addition to the IID analysis, the Restart Test defined in NIST SP 800-90B is performed. This test verifies that the entropy source does not produce identical or highly correlated outputs after multiple resets. The device is restarted several times, and a short sequence is collected after each restart. The sequences are then compared to ensure that no bit position shows excessive repetition. Figure 6 shows six bytes collected after six independent restarts of the proposed design. The differences observed among the collected sequences indicate variation in the entropy source's startup behavior.

TABLE I: Entropy estimation of chaotic maps using NIST SP 800-90B.

Test Category	Logistic	Tent
Chi-square Independence	$p = 0.9509$	$p = 0.4135$
Chi-square Goodness-of-fit	$p = 0.3592$	$p = 0.9767$
LRS Test	Pass	Pass
Min-entropy / bit	0.9984	0.9942
Min-entropy / byte	7.9872	7.9536

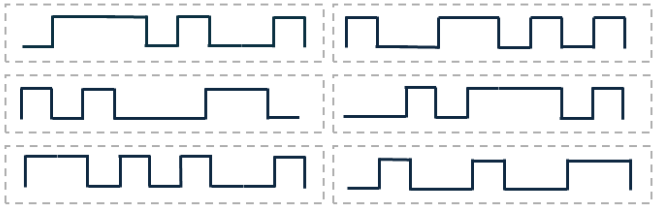


Fig. 6: Six output bytes collected after six independent restarts (Restart Test).

B. NIST SP 800-22 Statistical Randomness Evaluation

The NIST SP 800-22 test suite was used to assess the statistical randomness of the generated bitstreams [11]. As shown in Table II, both chaotic maps passed all tests with p-values above the required threshold. While both sequences exhibit valid randomness, the Logistic map consistently achieved higher p-values and pass rates, indicating slightly stronger statistical robustness.

C. AIS 31 Statistical Test Results

The AIS 31 test suite is used to evaluate the cryptographic quality of the generated sequences. As summarized in Table III, both the Logistic and Tent maps passed all basic tests (T0–T5) and achieved near-ideal results in the statistical tests (T6a–T8), with probabilities close to 0.5 and

Authorized licensed use limited to: NATIONAL INSTITUTE OF TECHNOLOGY KURUKSHETRA. Downloaded on October 01,2026 at 10:49:06 UTC from IEEE Xplore. Restrictions apply.

TABLE II: NIST SP800-22 statistical test results for chaotic maps

Randomness Test	Logistic Map		Tent Map	
	p-value	Rate	p-value	Rate
Frequency	0.5142	100%	0.5101	96.9%
Block Frequency	0.5341	100%	0.4876	96.9%
Cumulative Sums	0.4071	100%	0.3614	96.9%
Runs	0.8623	100%	0.8015	100%
Longest Run	0.0352	100%	0.0298	96.9%
Rank	0.7399	100%	0.7024	96.9%
FFT	0.5341	100%	0.4987	96.9%
Non-overlapping Template	0.4505	96.9%	0.4122	93.8%
Overlapping Template	0.8623	100%	0.7931	96.9%
Approximate Entropy	0.0669	96.9%	0.0524	93.8%
Serial	0.4071	100%	0.3620	96.9%
Linear Complexity	0.1005	96.9%	0.0821	93.8%

TABLE III: AIS 31 Statistical Test Results for Chaotic Maps

ID	Name	Logistic Map	Tent Map
T0	Disjointness	Passed	Passed
T1	Monobit	Passed	Passed
T2	Poker	Passed	Passed
T3	Run	Passed	Passed
T4	Long Run	Passed	Passed
T5	Autocorrelation	Passed	Passed
T6a	Uniformity	Mean = 0.5005	Mean = 0.5002
T6b	Transition	Prob = 0.5008	Prob = 0.5011
T7	Multinomial	Stat = 0.6919/0.6480	Stat = 0.7012/0.6541
T8	Entropy	7.9999 bits/byte	7.9998 bits/byte

TABLE IV: Comparison Results

Work	Entropy Source	FPGA	LUT	DFF	Throughput (Mbps)
[3]	Jitter	Spartan-6	56	19	100
[12]	Jitter	Virtex-5	320	320	200
[13]	Metastability	Cyclone IV	298	-	150
[14]	Metastability	Spartan-6	36	0	12.5
[15]	Metastability	Virtex-6	14	6	25
Proposed (Logistic)	Jitter	Zynq-7000	56	14	142.5
Proposed (Tent)	Jitter	Zynq-7000	42	10	142.5

entropy approaching 8 bits per byte. These results confirm the suitability of both maps for cryptographic randomness generation.

D. Hardware Utilization and Comparison

The design demonstrates low hardware overhead for the TRNG architecture, requiring only 56 LUTs and 14 DFFs for the Logistic map, and 42 LUTs and 10 DFFs for the Tent map implementation, as shown in Table IV. This utilization accounts only for the TRNG core and does not include resources required for bitstream transmission.

Table IV also compares the proposed TRNG with existing FPGA-based designs. The proposed architecture achieves high throughput while maintaining low hardware utilization. For instance, compared to [12], which targets maximum throughput but requires considerably more hardware resources, the proposed TRNG achieves strong performance with a much lower implementation cost. On the other hand, although [14] focuses on minimizing resource usage, this comes at the expense of throughput. The proposed design achieves a more practical balance between these two extremes. Overall, it provides an efficient compromise between performance and hardware complexity, making it attractive for resource-constrained FPGA platforms.

V. Conclusion

This paper presented a TRNG architecture that combines Fibonacci ring oscillators with noise-perturbed Logistic and Tent chaotic maps on a Zynq-7000 Zybo FPGA. A lightweight correction scheme further reduced bias without degrading throughput. The design enhanced entropy while maintaining low hardware complexity. The Logistic map achieved higher statistical robustness with a min-entropy of 0.9984 bits/bit, whereas the Tent map improved hardware efficiency with fewer resources. Experimental results confirmed compliance with NIST SP 800-90B, NIST SP 800-22, and AIS 31 standards. Future work will explore parallel architectures and system-level integration.

Acknowledgments

This work is supported by the Science, Technology and Development Fund (STDF) under grant number (#50816).

References

[1] Y. Chen, Y. Tian, R. Zhou, D. M. Castro, D. Guo, and Q. Zhou, "NDSTRNG: Non-deterministic sampling-based true random number generator on soc FPGA systems," *IEEE Transactions on Computers*, vol. 73, no. 5, pp. 1313–1326, 2024.

[2] M. Drutarovsky and P. Galajda, "A robust chaos-based true random number generator embedded in reconfigurable switched-capacitor hardware," in *2007 17th International Conference Radioelektronika*, pp. 1–6, IEEE, 2007.

[3] X. Wang, H. Liang, Y. Wang, L. Yao, Y. Guo, M. Yi, Z. Huang, H. Qi, and Y. Lu, "High-throughput portable true random number generator based on jitter-latch structure," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 68, no. 2, pp. 741–750, 2020.

[4] H. O. Ahmed, "MCLM-BXN: Design of a high-entropy TRNG using modified chaotic logistic maps and braided XOR networks on FPGA," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2025.

[5] H. O. Ahmed, D. Kim, and W. J. Buchanan, "A true random number generator based on race hazard and jitter of braided and cross-coupled logic gates using FPGA," *IEEE Access*, 2024.

[6] F. Frustaci, F. Spagnolo, S. Perri, and P. Corsonello, "A high-speed FPGA-based true random number generator using metastability with clock managers," *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 70, no. 2, pp. 756–760, 2022.

[7] F. Frustaci, F. Spagnolo, P. Corsonello, and S. Perri, "A high-speed and low-power DSP-based TRNG for FPGA implementations," *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 71, no. 12, pp. 4964–4968, 2024.

[8] P. Nannipieri, S. Di Matteo, L. Baldanzi, L. Crocetti, J. Belli, L. Fanucci, and S. Saponara, "True random number generator based on fibonacci-galois ring oscillators for FPGA," *Applied Sciences*, vol. 11, no. 8, p. 3330, 2021.

[9] S. M. Mohamed, W. S. Sayed, L. A. Said, and A. G. Radwan, "Reconfigurable fpga realization of fractional-order chaotic systems," *IEEE Access*, vol. 9, p. 89376–89389, 2021.

[10] M. Sönmez Turan, E. Barker, J. Kelsey, K. McKay, M. Baish, and M. Boyle, "Recommendation for the entropy sources used for random bit generation," tech. rep., National Institute of Standards and Technology, 2016.

[11] A. Rukhin, J. Soto, J. Nechvatal, M. Smid, and E. Barker, "A statistical test suite for random and pseudorandom number generators for cryptographic applications," tech. rep., 2001.

[12] A. Cherkaoui, V. Fischer, L. Fesquet, and A. Aubert, "A very high speed true random number generator with entropy assessment," in *International conference on cryptographic hardware and embedded systems*, pp. 179–196, Springer, 2013.

[13] X. Wu and S. Li, "A new digital true random number generator based on delay chain feedback loop," in *2017 IEEE International Symposium on Circuits and Systems (ISCAS)*, pp. 1–4, IEEE, 2017.

[14] R. Della Sala, D. Bellizia, and G. Scotti, "High-throughput FPGA-compatible TRNG architecture exploiting multistimuli metastable cells," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 69, no. 12, pp. 4886–4897, 2022.

[15] Q. Peng, J. Bian, Z. Huang, S. Wang, and A. Yan, "A compact TRNG design for FPGA based on the metastability of RO-driven shift registers," *ACM Transactions on Design Automation of Electronic Systems*, vol. 29, no. 1, pp. 1–17, 2023.

Authorized licensed use limited to: NATIONAL INSTITUTE OF TECHNOLOGY KURUKSHETRA. Downloaded on October 01,2026 at 10:49:06 UTC from IEEE Xplore. Restrictions apply.